PORTFOLIO PROJECT

CyberAudit & Solutions

SME Audit Management Platform

Stage 3: Technical Documentation

Holberton School Dijon | 2026

Tommy JOUHANS : Front-End Developer
James ROUSSEL : Back-End Developer

Summary

Introduction

1.1 Context

CyberAudit & Solutions is a full-stack web platform that connects French SMEs/VSEs without an internal IT team with cybersecurity experts. According to ANSSI 2023, 45% of SMEs have no dedicated IT team and remain exposed to cyberattacks (ransomware, phishing, data breaches).

1.2 Scope of the document

This technical documentation covers the detailed design of the MVP. It describes:

- User needs formalized into prioritized User Stories
- The software architecture and data flow; the data model (relational PostgreSQL / MySQL)
- The back-end classes, front-end components, interaction scenarios, and exposed and consumed APIs
- The code management and quality strategy

1.3 Target audience

- Development team (Tommy + James): reference during the coding phase (W2 → W12).
- Holberton Jury: proof of the detailed MVP design.
- RNCP 5 Jury: reusable skills portfolio for Web and Mobile Web Developer certification.

1.4 Stack technique

Layer	Technology	Justification
Front-End	React + Tailwind CSS	High-performance SPA, rapid design system
Back-End	Django + Django REST Framework (DRF)	Native security, mature ecosystem
Database	PostgreSQL or MySQL	ACID relational analysis, high-performance indexes
Async	Celery + Redis	Long tasks to be developed beyond the RNCP level 5 web and mobile web developer
Auth	SimpleJWT	Stateless, scalable
Accommodation	Vercel + Railway	HTTPS auto, déploiement Git

User Stories & Mockups

2.1 Personas

Marie: 45 years old, manager of an accounting firm in Dijon, no IT knowledge. UX target < 3 min.

Karim: 32 years old, cybersecurity expert, manages the request queue in parallel.

2.2 User Storie priorisée

User Story	Stage W (Week)
As a visitor, I want to register in order to create my account.	**W3-4**
As a user, I want to log in to access my account.	**W4**
As a user, I want to log out in order to protect my account.	**W5**
As an SME customer, I want to submit a request in < 3 min.	**W6**
As a customer, I want to track the status (4 statuses).	**W7**
As a customer, I want to receive an email for every change.	**W8**
As an admin, I want to see all requests sorted.	**W9**
As an admin, I want to change the status.	**W10**
As an admin, I want to generate a PDF.	**W10**
As a customer, I want to download my PDF.	**W10**

Should Have

- Awareness modules
- Password reset
- Admin filters
- UI notifications.

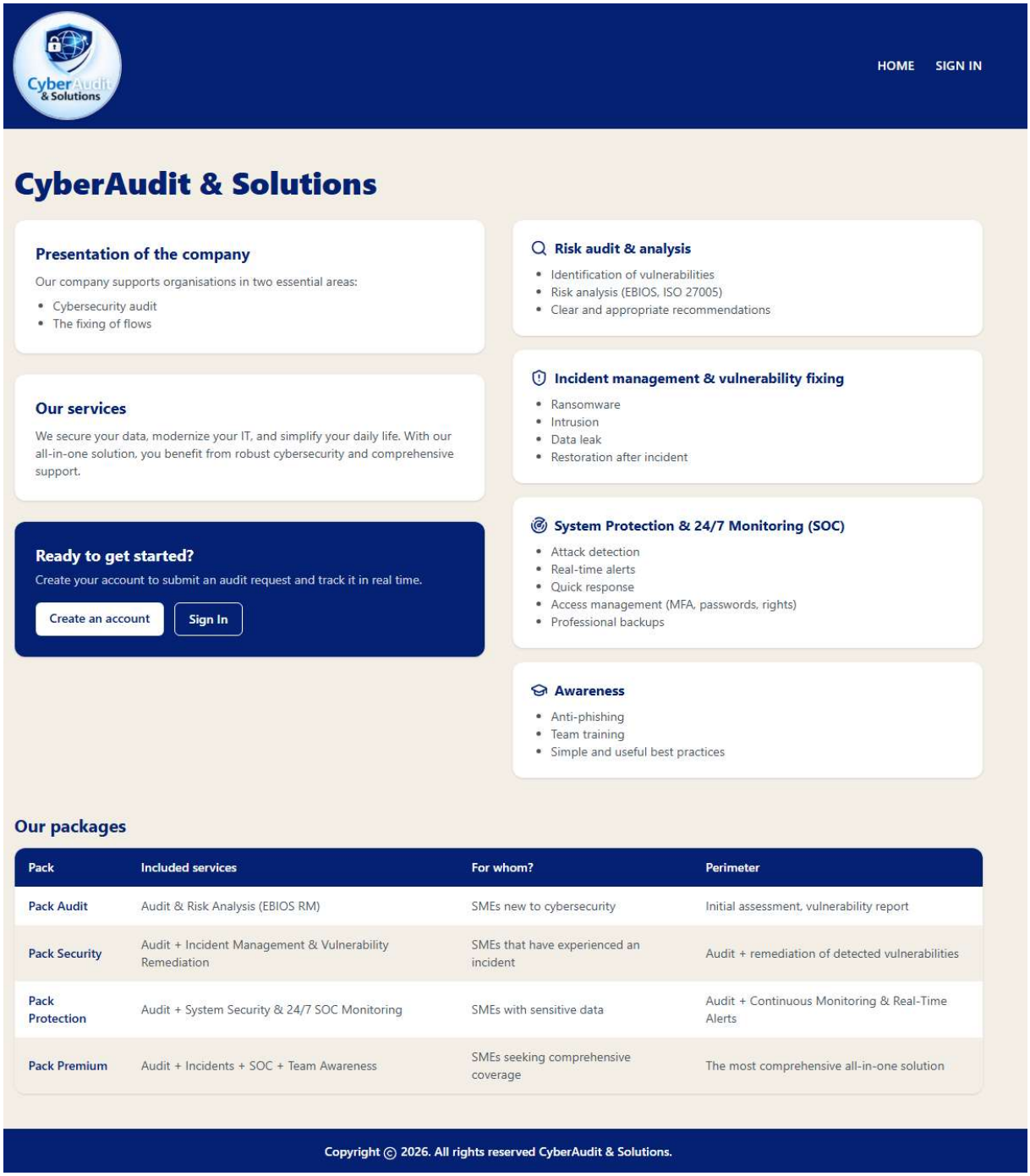
Could Have / Won't Have
Outside the scope:

- mobile app
- AI scoring
- Marketplace
- OAuth2
- gamification

2.3 Screen Descriptions (11 Figma mockups to be produced in W1)

Screen 1: Public Landing

The CyberAudit & Solutions platform homepage presents the company, its services, and the packages offered, tailored to entrepreneurs with fewer than 50 employees. Simply go to the "SIGN IN" tab so the client (Marie) can create an account and log in.

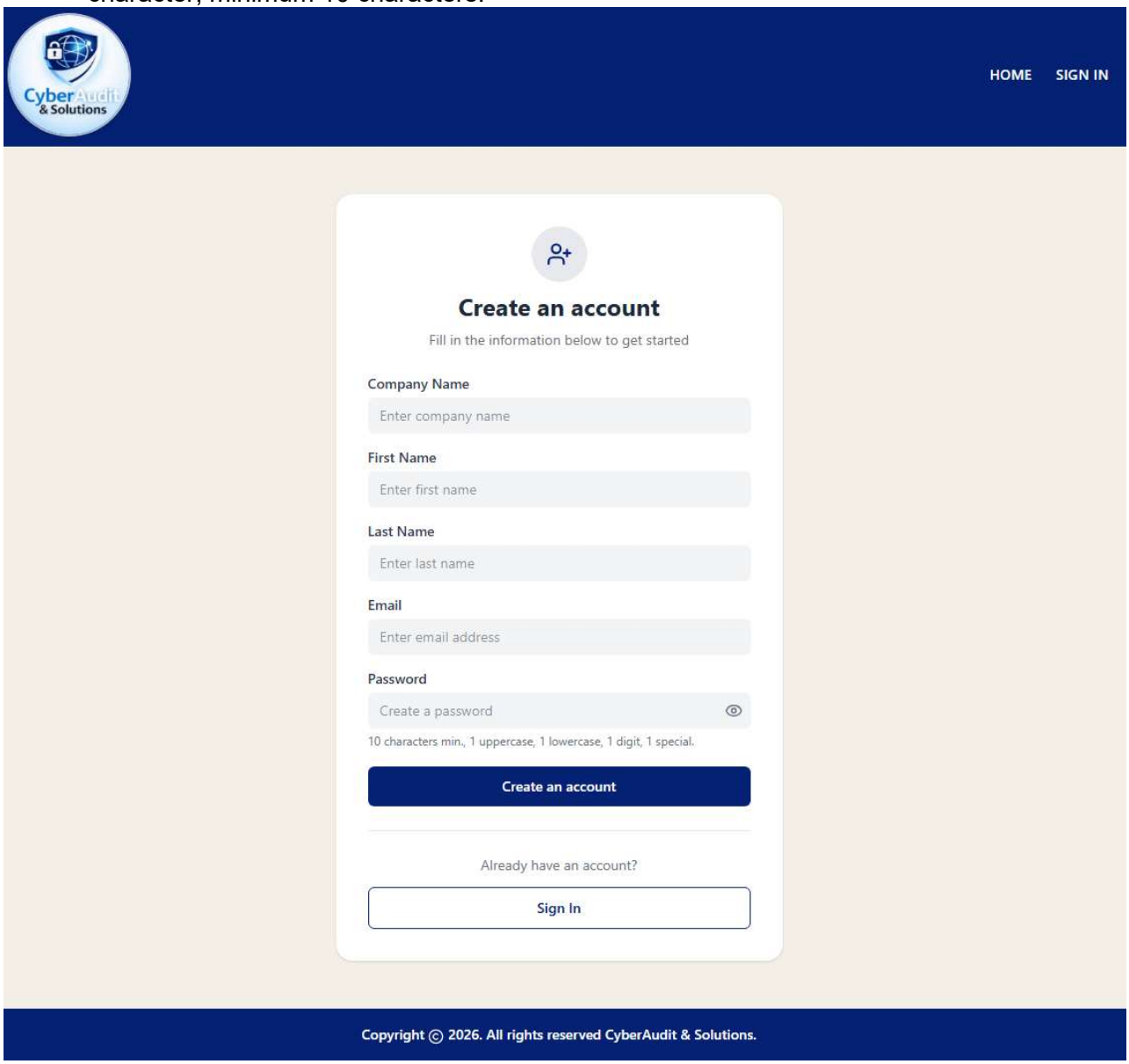


Screen 2: Registration

After clicking "Create an account," the customer enters her company name, first name, last name, email address, and password (hashed).

Form completion rules:

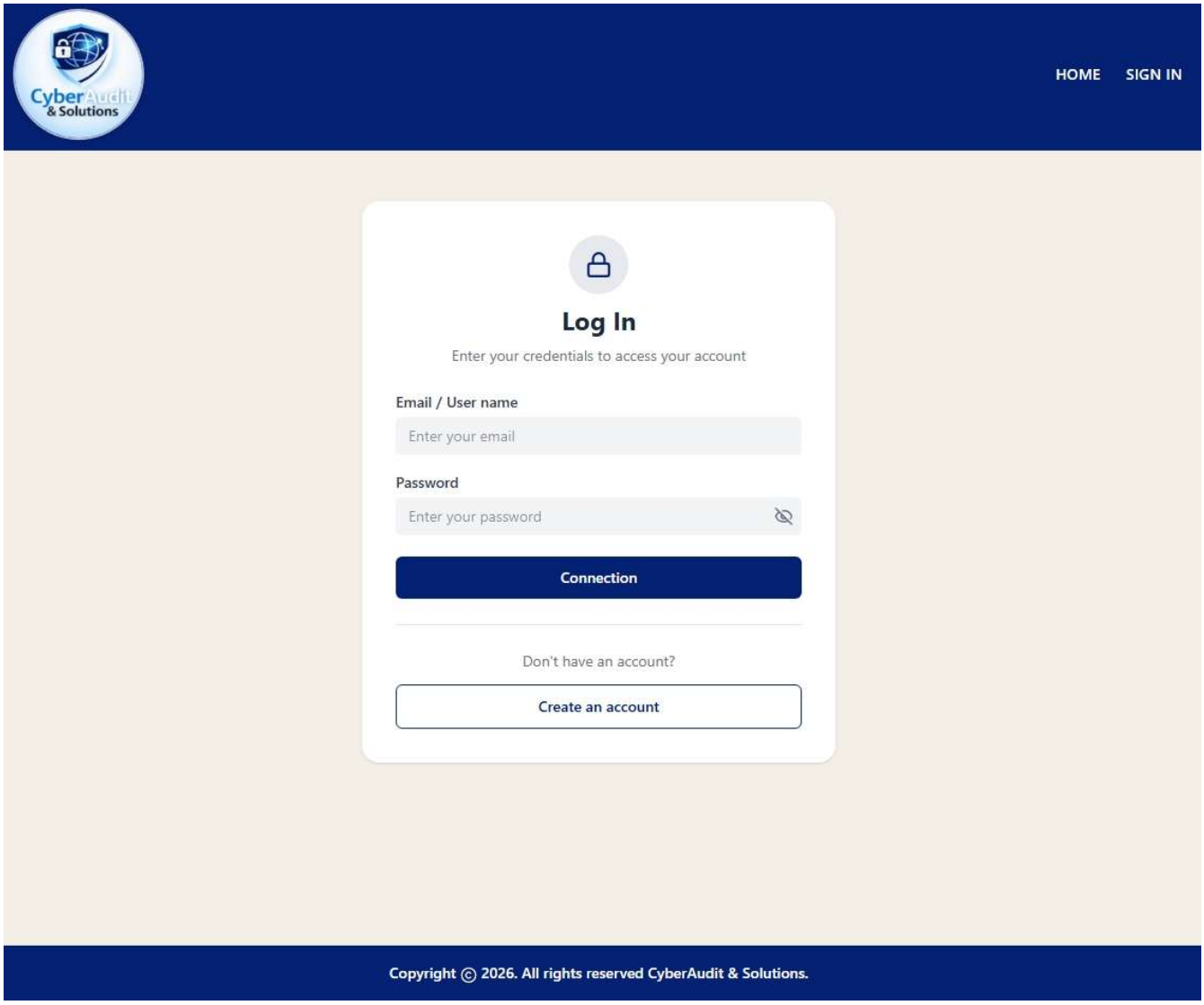
- Each field contains a maximum of 50 characters.
- If a field is missing error message "Please enter the information on the form."
- Email address format: name@domain required.
- Password: one lowercase letter, one uppercase letter, one number, one special character, minimum 10 characters.



Screen 3: Login

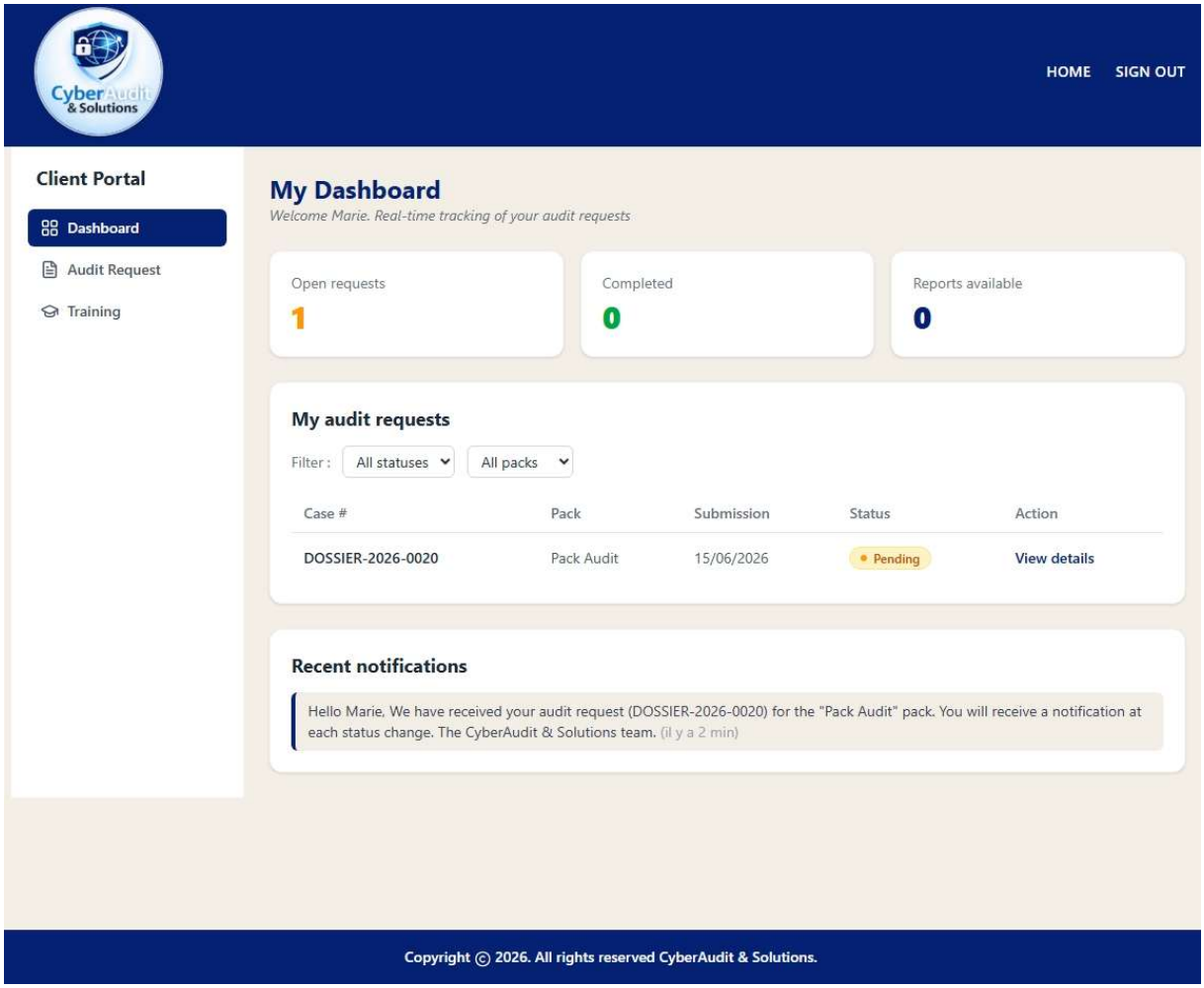
The customer can log in with their account; otherwise, they click on "Create an account."

"Lost Mail" and "Lost Password" links are planned (to be developed later for the presentation RNCP 5).



Screen 4: Client Dashboard

When the client is logged in, she can view her dashboard to track the progress of her audit request (Pending / In Progress / Completed). She can click on "View details" and download the report via "Download report".

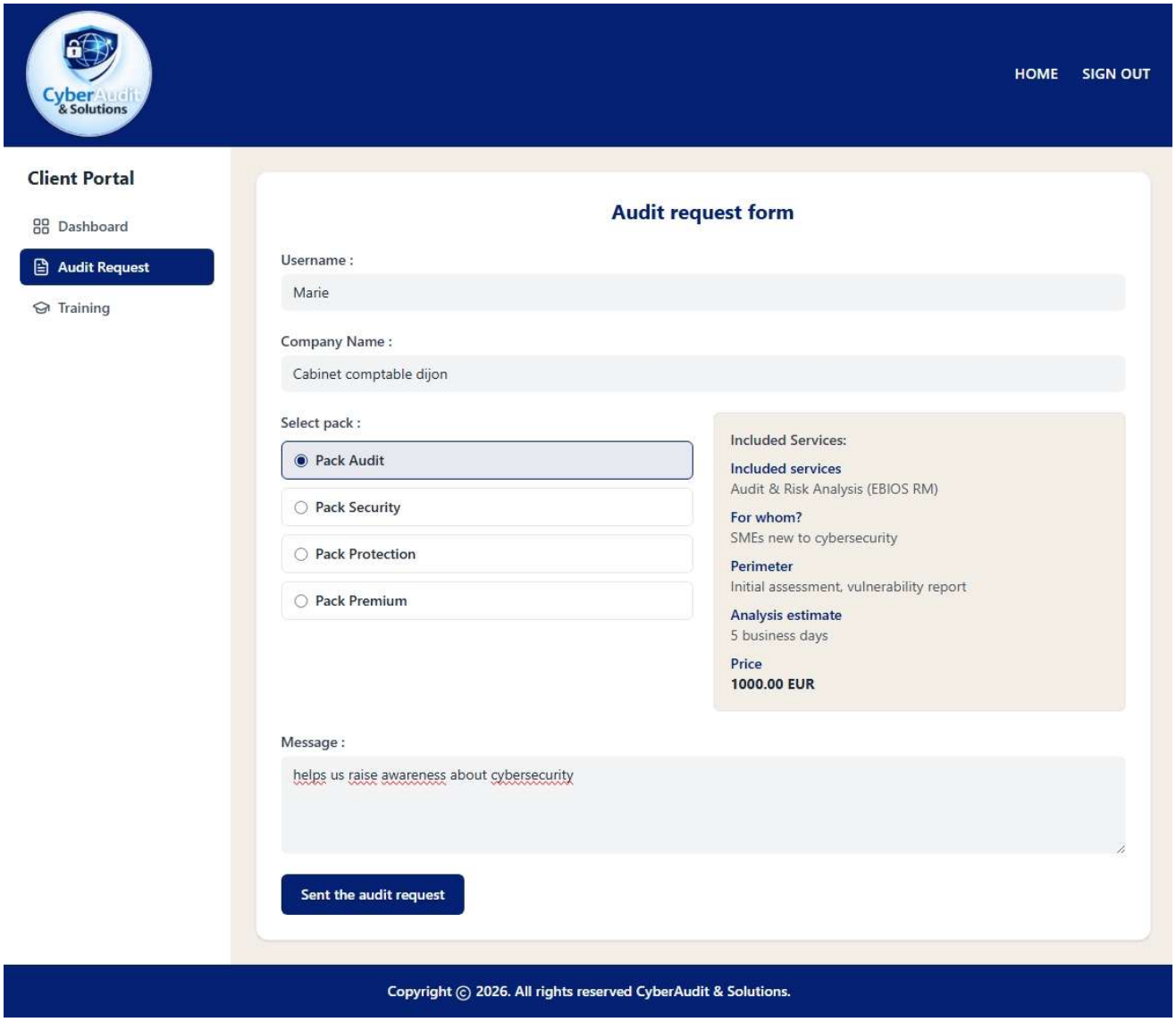


Screen 5: Audit Form

By clicking the "Audit Request" tab, the client accesses the audit request form. She must fill in:

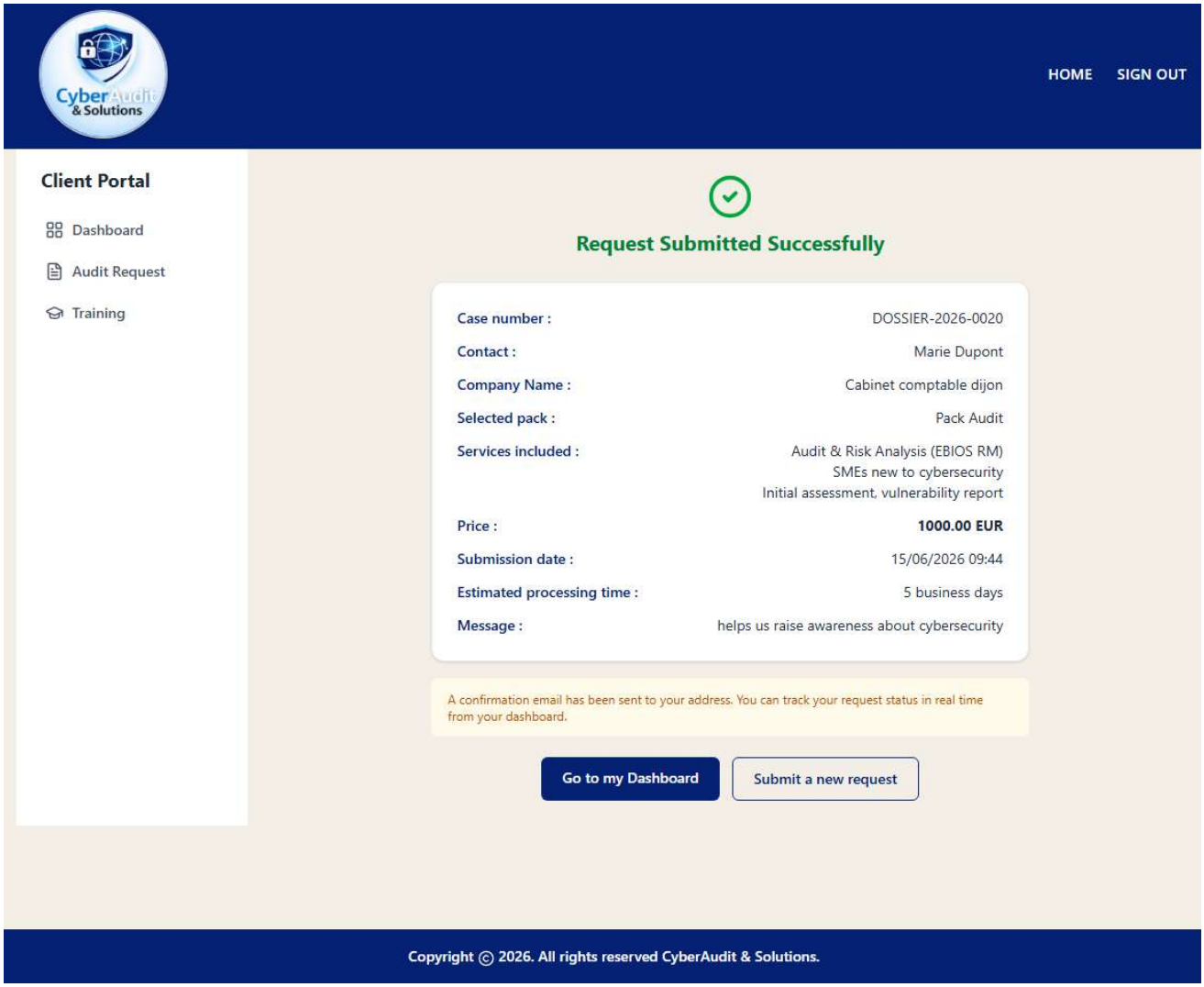
- Her username (maximum 50 characters)
- The company name (less than 50 characters)
- Choose the package (Audit / Security / Protection / Premium). When the client selects the package using the radio button, the package details are displayed in the "Includes Services" panel, including the "Included Services," "For whom and what needs are being addressed?", "Scope of analysis," "Analysis estimate," and "Price."
- An optional message describing the desired audit request.

After completing the form, the client clicks "Send the Audit Request" to submit the audit request.



Screen 6: Request details (client)

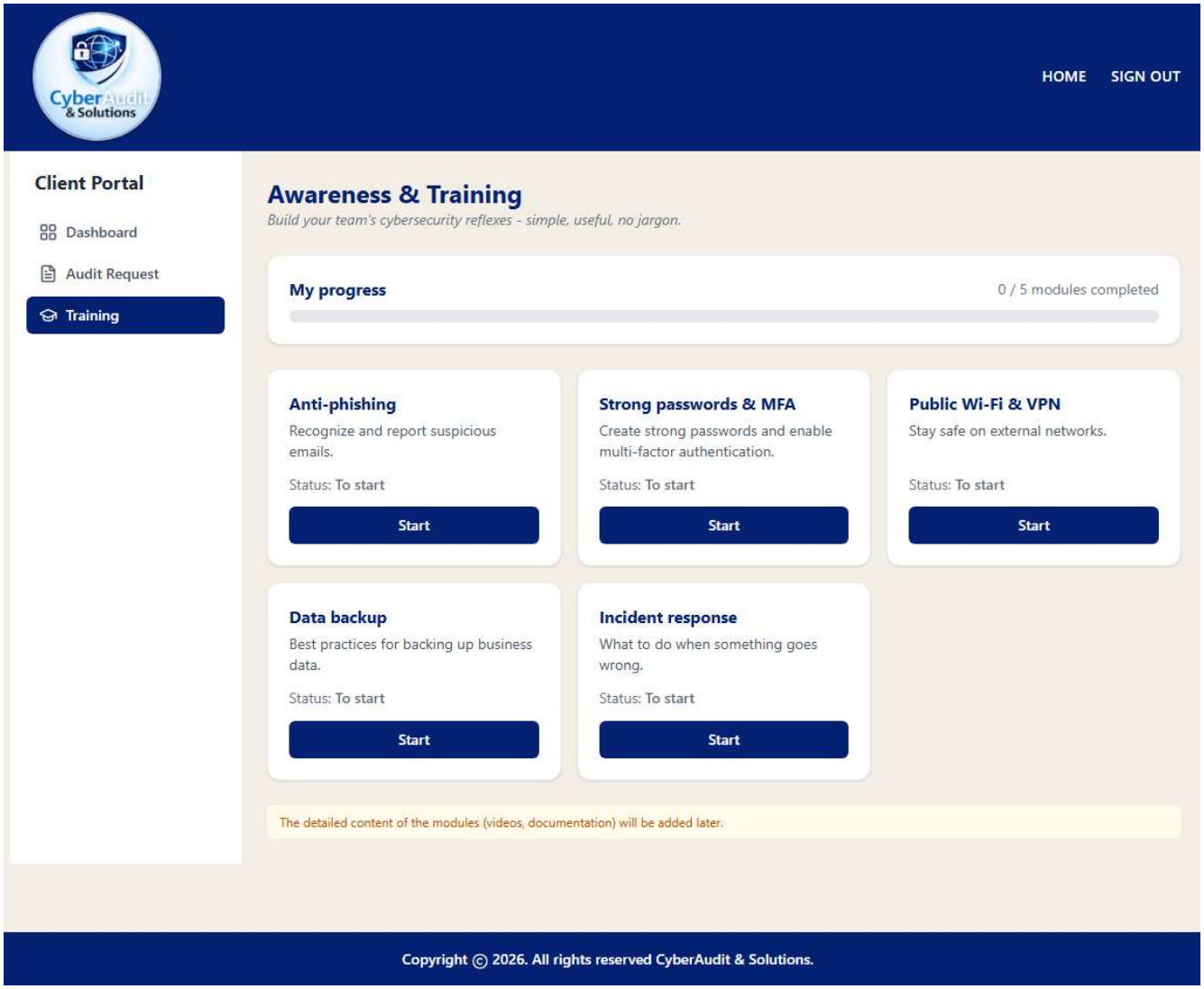
After sending, confirmation message to the expert (Karim) and acknowledgment of receipt to the client. The client follows the progress of the analysis.



Screen 7: Training Module

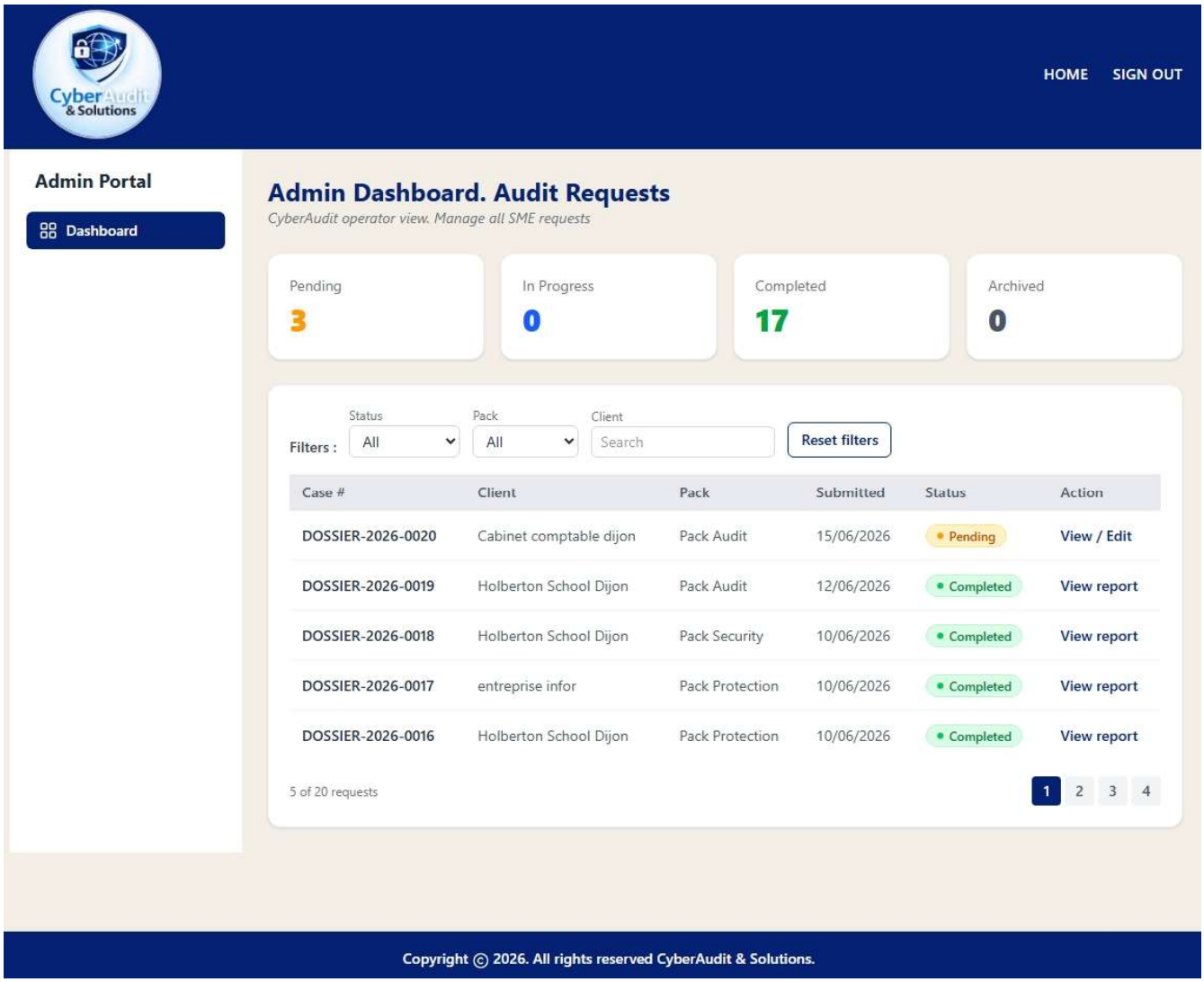
To be developed later. Modules: 'Anti-phishing', 'Multi-factor Authentication', 'Public Connection and Wi-Fi & VPN', 'Data Backup', 'Incident Response'. In various formats (video, video conference, documentation) with a progress bar.

This part of the section will be presented in the future beyond the RCNP 5 Web and Mobile Developer title. The CyberAudit & Solutions team should help us prepare the training modules in the application.



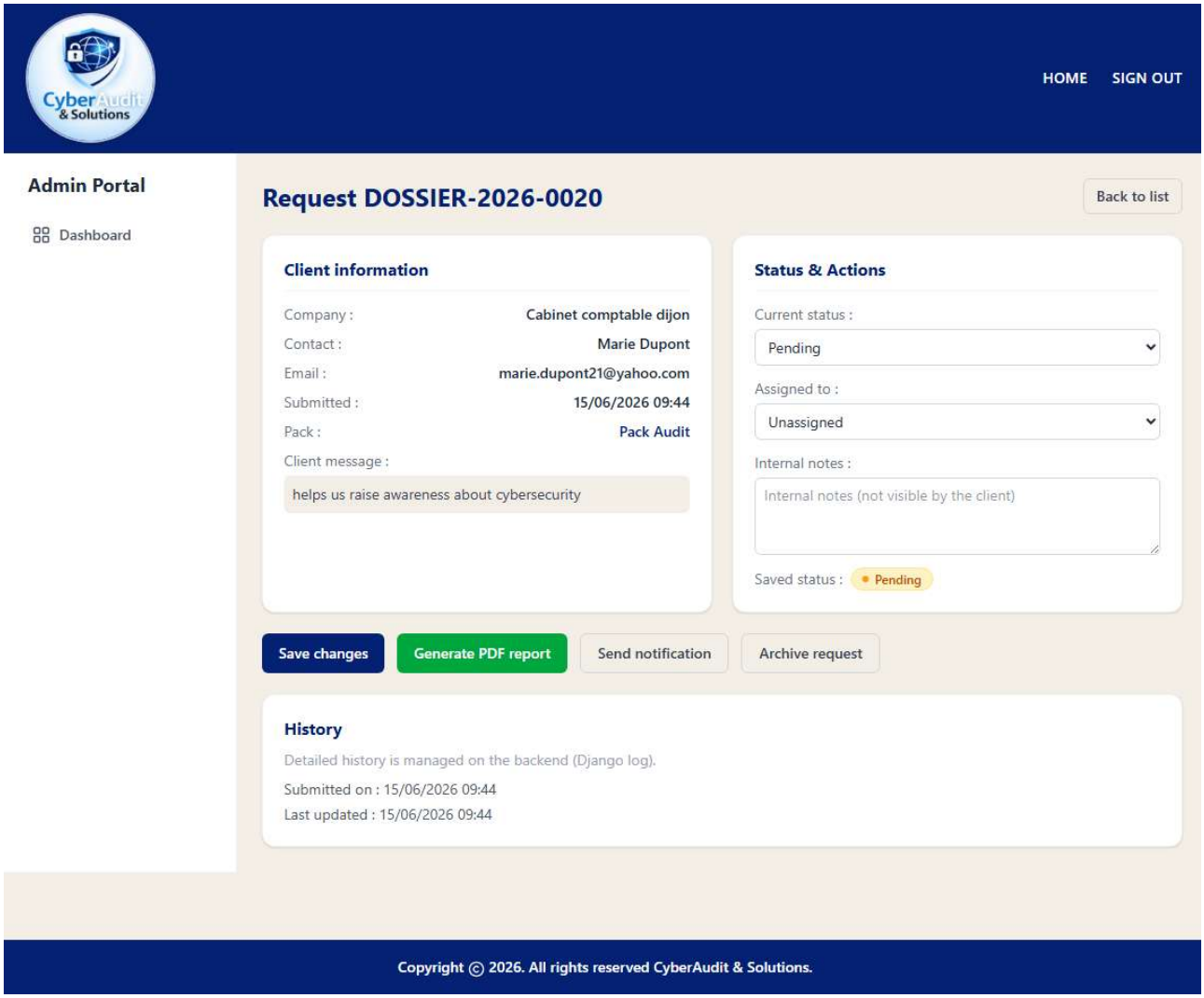
Screen 8: Admin Dashboard

The expert (Karim) accesses his dashboard with all the requests. He can sort the list according to the chosen package and the current statuses, then view and edit each request.



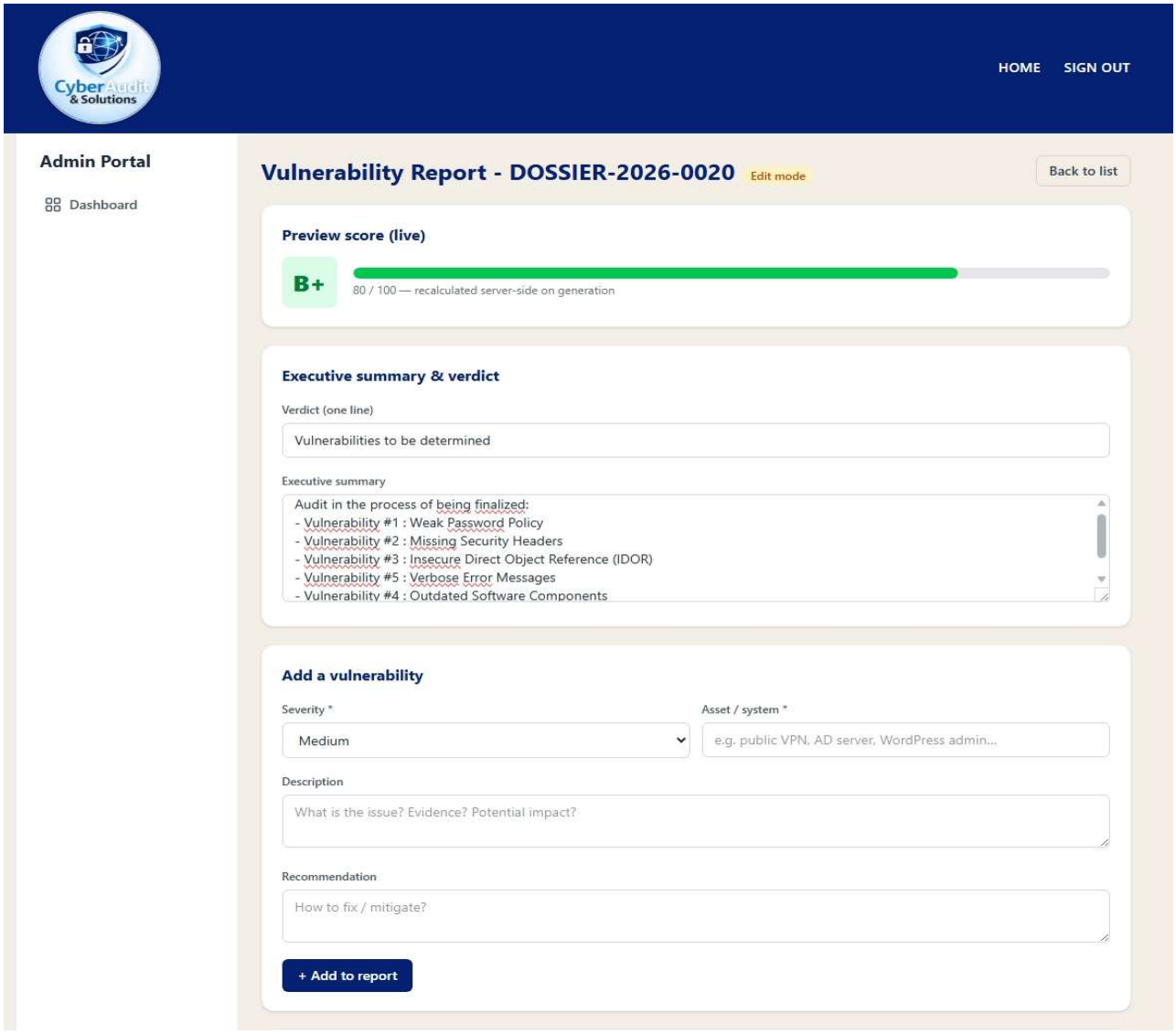
Screen 9: Request Detail (admin)

By clicking on "View/edit," the expert can view the client's request, change the status via the "Status & Actions" form, generate a PDF report via "Generate PDF report," send a notification to the client ("Send notification"), and can archive the request ("Archive request").



Screen 10: PDF report viewer

Report can be downloaded. The report identifies:
- The level of vulnerabilities (Low, Medium, High)
- The assets (mail server, wifi router, etc.)
- The description and the recommendations
- An action plan is generated with the estimated number of days.



Vulnerability Report - DOSSIER-2026-0020

Back to list | Download PDF

Global security score

B+

Vulnerabilities to be determined

80 / 100

Executive summary

Audit in the process of being finalized:
- Vulnerability #1 : Weak Password Policy
- Vulnerability #2 : Missing Security Headers
- Vulnerability #3 : Insecure Direct Object Reference (IDOR)
- Vulnerability #5 : Verbose Error Messages
- Vulnerability #4 : Outdated Software Components

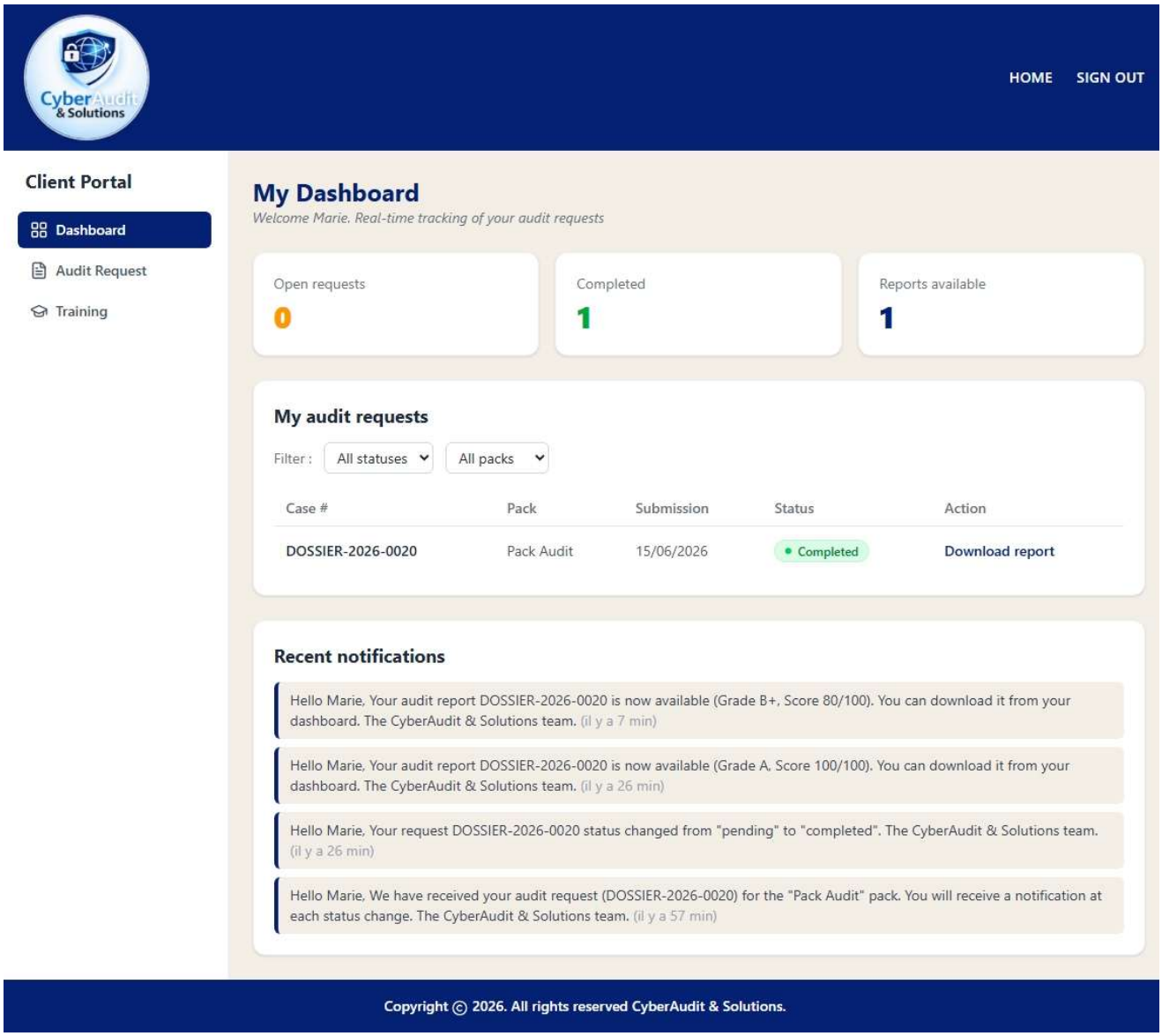
Vulnerabilities identified (5)

Severity	Asset	Description	Recommendation
Medium	Web Application - Authentication Module	The application accepts weak passwords without enforcing minimum complexity requirements. Evidence: A user account was successfully created using the password "password123". Potential Impact: Attackers may compromise user accounts through brute-force or dictionary attacks	Enforce a strong password policy, including minimum length, complexity requirements, and checks against known compromised passwords. Consider implementing multi-factor authentication (MFA).
Low	Web Frontend	Several recommended HTTP security headers are missing, including Content-Security-Policy (CSP), X-Frame-Options, and X-Content-Type-Options. Evidence: Inspection of HTTP responses revealed the absence of these headers. Potential Impact: Increased exposure to attacks such as clickjacking and content injection.	Configure appropriate HTTP security headers across all application responses and verify their implementation through security testing.
High	REST API - User Management	Authenticated users can access resources belonging to other users by modifying object identifiers within API requests. Evidence: Accessing /api/users/123 returned information associated with another account without proper authorization checks. Potential Impact: Unauthorized disclosure of sensitive user information and potential privacy violations.	Implement server-side authorization checks to ensure users can only access resources they are authorized to view or modify.
Medium	Application Server	The application relies on outdated third-party libraries containing known security vulnerabilities. Evidence: Dependency analysis identified unsupported versions with publicly disclosed CVEs. Potential Impact: Exploitation of known vulnerabilities could lead to unauthorized access, data compromise, or denial of service.	Update all affected dependencies to supported versions and establish a regular patch management process.
Low	Web Application	Detailed application error messages disclose internal implementation details, including stack traces and framework information. Evidence: Triggering invalid requests resulted in detailed exception messages being displayed. Potential Impact: Information disclosure may assist attackers in identifying technologies and attack vectors.	Replace detailed error messages with generic user-facing messages and log technical details securely on the server side.

Edit report

Screen 11: Download from the dashboard

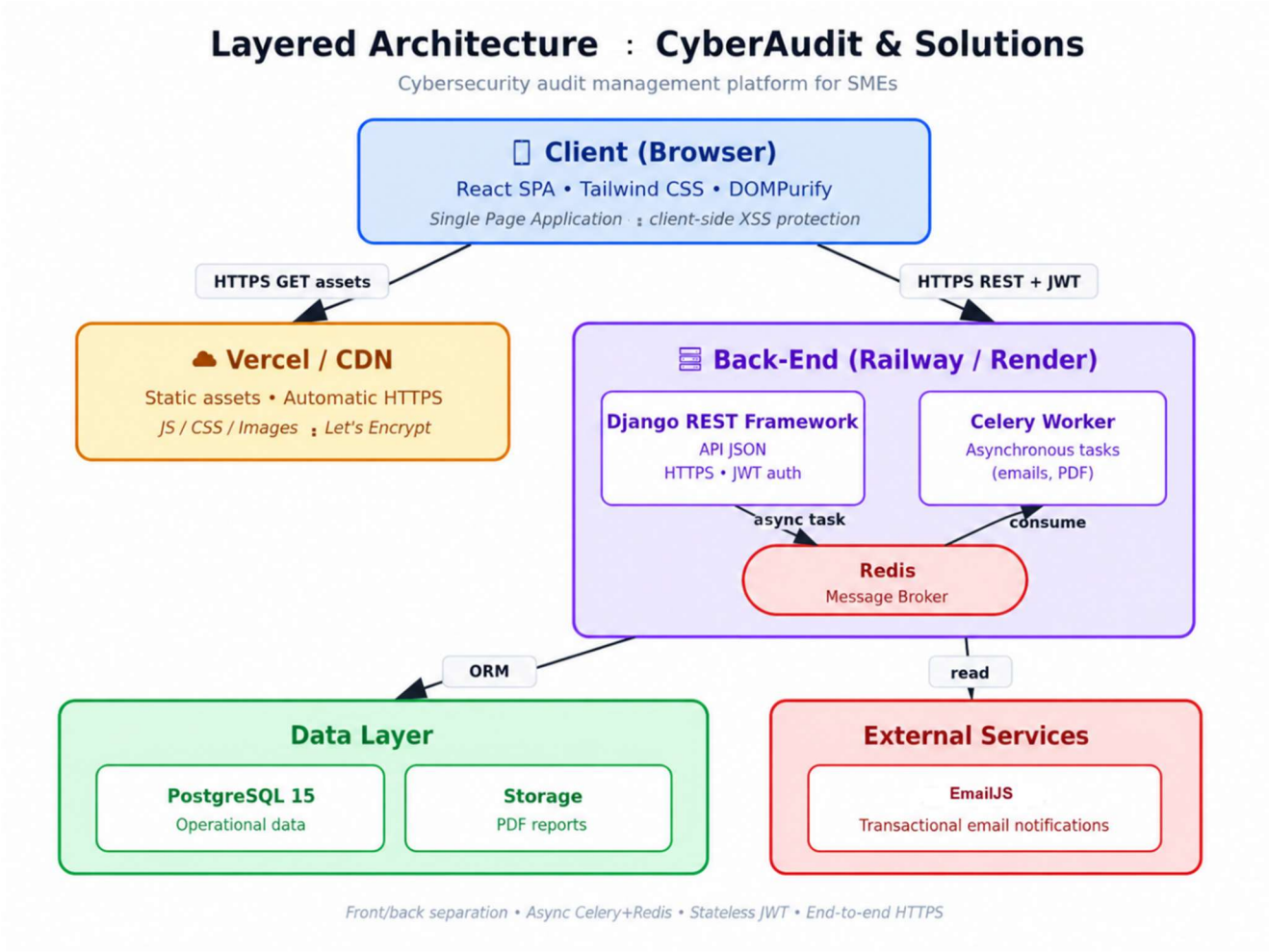
The client can download the report from their dashboard via the "Download report" link.



System architecture

3.1 Overview: Layered Architecture

The architecture follows a layered model:



Layered architecture:
Client (browser) → CDN/Backend → Data + External Services.
Client (Browser):

- React SPA
- Tailwind CSS
- DOMPurify (anti-XSS)
- HTTPS
- REST + JWT

Vercel / CDN :

- Static assets
- HTTPS Let's Encrypt

Back-End (Railway / Render):
- Django REST API (JSON, JWT, HTTPS)
- Celery Worker (async)
- Redis (broker)

- ORM
- SMTP

Data layer: PostgreSQL or MySQL + PDF Storage.

External services : Gmail SMTP (email notifications).

3.2 Data Flow: Audit Request

Client fills out the form in the React SPA.

The front end sends POST /api/audits/ (JSON + JWT in Authorization: Bearer).

Django validates the request (DRF serializer) and persists it in the database via the ORM.

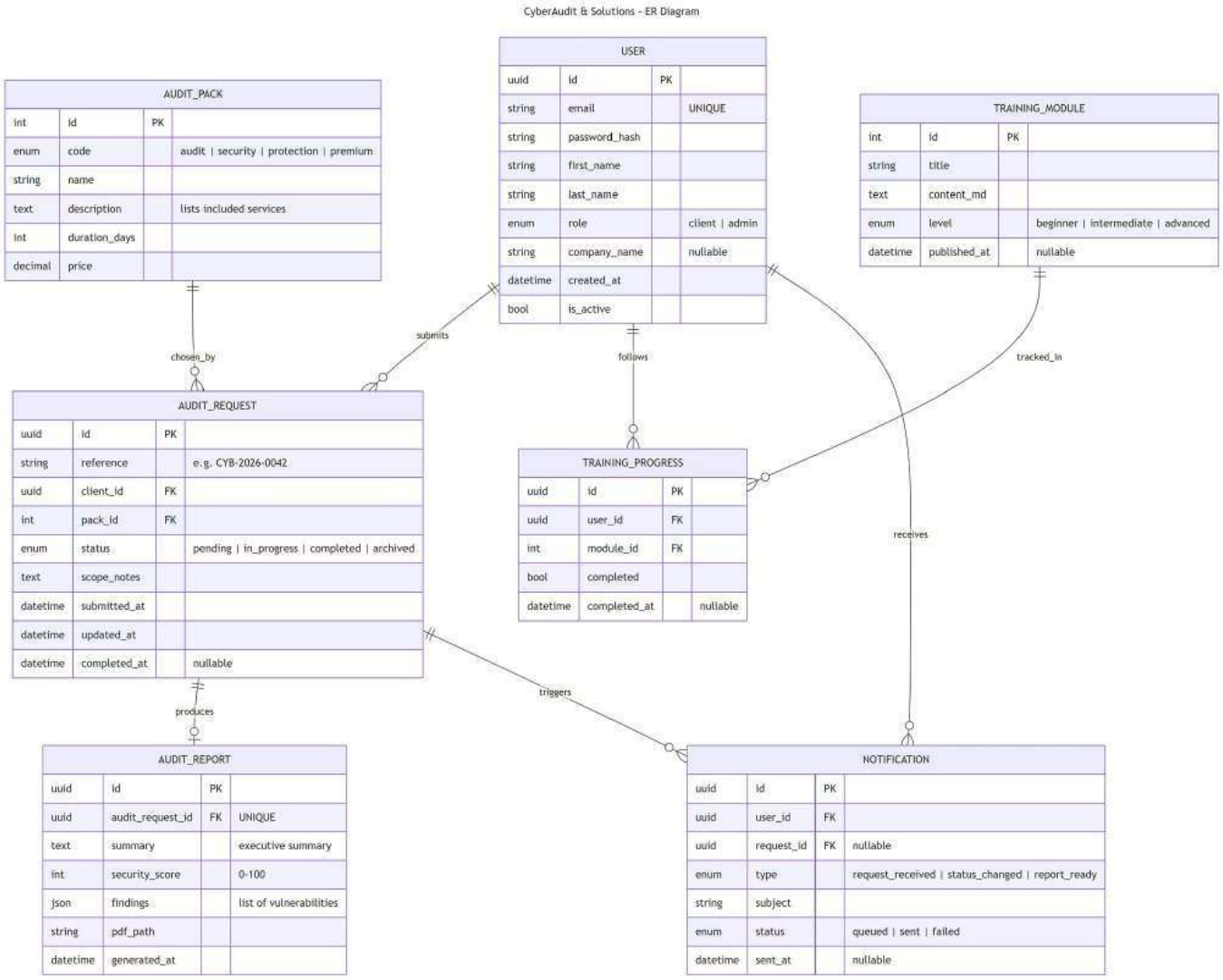
Django publishes a Celery task send_acknowledgment_email(audit_id) in Redis.

3.3 Justified architecture choices

- **Front/back decoupling** → 2 independent deployments, controlled CORS, cacheable SPA on CDN.
- **Async Celery** → generating a PDF can take 5-10 s; the HTTP thread is not blocked.
- **PostgreSQL relational** → highly structured data with relationships (User → AuditRequest → Report), critical ACID integrity.
- **JWT stateless** → no server session, horizontally scalable

Components, classes and DB

4.1 Database Schema



Entity	Key fields	Relationship
USER	id (UUID), email (UK), password_hash, first_name, last_name, role (client/admin), company_name, created_at, is_active	1→N AuditRequest (submits / follows / receives)
AUDIT_PACK	id, reference, code (audit/security/protection/premium), name, description, duration_days, price	1→N AuditRequest (chosen_by)
AUDIT_REQUEST	id (UUID), client_id (FK), pack_id (FK), status (pending/ in_progress/done/archived), scope_notes, submitted_at, updated_at, completed_at	N→1 User+Pack ; 1→1 Report ; 1→N Notification
AUDIT_REPORT	id (UUID), audit_request_id (FK UK), pdf_path, summary, findings (JSON), generated_at	1→1 AuditRequest
TRAINING_MODULE	id, title, content_md, duration_min, level, published_at	1→N Progress
TRAINING_PROGRESS	id, user_id, module_id, completed, started_at, completed_at	N→1 User + Module
NOTIFICATION	id, user_id, request_id, type, subject, status, sent_at	N→1 User + Request

Recommended indexes:

- USER.email UNIQUE
- AUDIT_REQUEST.status BTREE (frequent admin filtering)
- (AUDIT_REQUEST.client_id, AUDIT_REQUEST.created_at) composite (client dashboard)
- AUDIT_REPORT.audit_request_id UNIQUE (1 report per request)

4.2 Back-end class diagram (Django models + services)

Class User

Represents a platform user, whether they are an SME client or an administrator. Manages JWT authentication and role separation.

Attribute	Description
id : UUID	Unique identifier (PK)
email : str	Email address (unique, indexed)
_password_hash : str	Hashed password (PBKDF2)
first_name, last_name	First and last name
role : enum	Client or admin
company_name : str	Company name (nullable)
is_active : bool	Account activated or deactivated
created_at : datetime	Account creation date

AuditPack Class

Represents one of the 4 packs (Audit, Security, Protection, Premium). Reference data loaded via luminaire.

Attribute	Description
id: int	Unique identifier
code: enum	audit / security / protection / premium
name: str	Commercial name
description: str	List of included services
duration_days: int	Incremental processing time
price: Decimal	price of the package

AuditRequest Class

Core of the business: the audit request submitted by an SME client.
Cycle of 4 statuses (pending → in_progress → completed → archived).

Attribute	Description
id: UUID	Unique identifier
reference: str	Case number (e.g., CYB-2026-0042)
client_id: UUID FK → User	SME Client who submitted
pack_id: UUID FK → AuditPack	Selected package
status: str(enum)	pending / in_progress / completed / archived
scope_notes: str	Form details
submitted_at : datatime	Cycle dates

Methods: generate_reference(), update_status(new_status), estimated_completion()

Class AuditReport

Vulnerability report linked to a single AuditRequest, includes a visual score and a downloadable PDF.

Attribute	Description
id: UUID	Unique identifier
audit_request_id: int FK	1-1 relationship
summary: str	Simplified executive summary
security_score: int	Score 0-100 (gauge)
findings: dict (JSON)	List of vulnerabilities
pdf_path: str	Path to generated PDF
generated_at : Datatime	Generation date

Methods: build_pdf(), get_download_url()

Class Notification

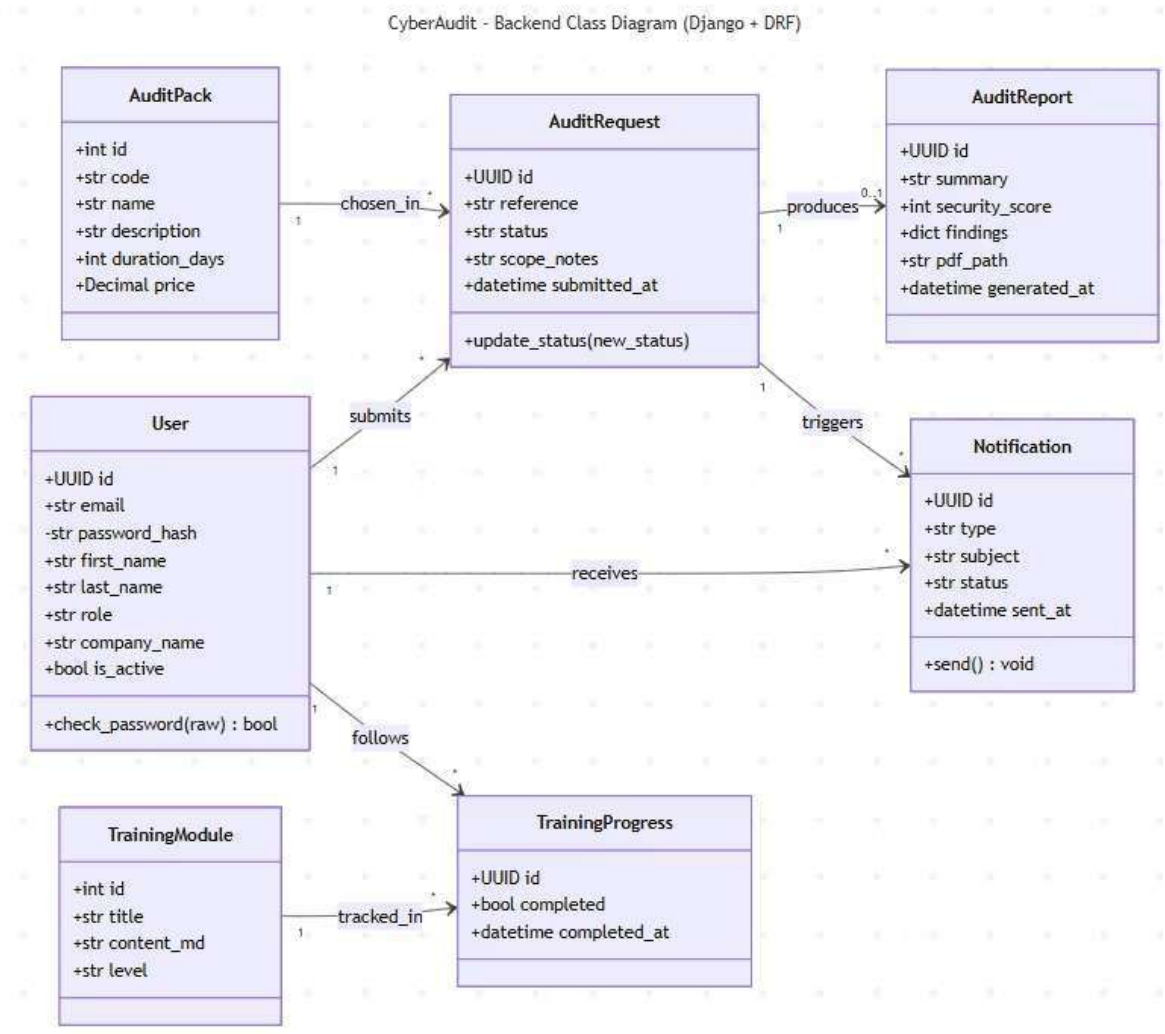
Trace toutes les communications e-mail envoyées via Celery.

Attribute	Description
id: UUID	Unique identifier
user_id: UUID FK → User	Recipient
request_id: UUID FK	Request concerned (nullable)
type: str(enum)	request_received / status_changed / report_ready
status: str(enum)	queued / sent / failed

Methods: send(), retry()

Classes TrainingModule & TrainingProgress

Cybersecurity awareness modules (phishing, MFA, GDPR…) with user progress tracking.Modules de sensibilisation cybersécurité (phishing, MFA, RGPD…) avec suivi de progression utilisateur.



4.3 Front-end components (React + Tailwind CSS)

The front-end is a React SPA communicating with the REST API via JWT. The site structure is organized into pages (full views), reusable UI components, a global store, and API calling services.

Authentification & routage

AuthStore : Global state (user, accessToken, refreshToken. Methods: login, logout, refresh, hasRole).

ProtectedRoute : Guard component, props (allowedRoles. Redirects to /login if not authenticated).

ApiClient : Axios wrapper that automatically adds the JWT to each request and handles the refresh token in case of a 401 error.

Public Pages

LoginPage : Login form. Submits to AuthStore.login() and then redirects according to the user's role.

RegisterPage : SME customer registration form. Calls AuthStore.register().

SME Customer Area

Client Dashboard: Dashboard listing the requests of the connected client.

AuditRequestForm: 3-step form (pack, scope, confirmation). Displays 4 Packcards.

RequestDetail: Detailed view with timeline, PDF report, and score.
Components: StatusBadge, SecurityScoreGauge, FindingsList.

CyberAudit Administrator Area

AdminDashboard: Paginated view of all requests with filters (status, package, date, client) and sorting.

Training module

TrainingPage: List of modules with progress indicator.

TrainingModuleViewer: Displays Markdown content. Security: cleanup via DOMPurify (antiXSS, SMART #3).

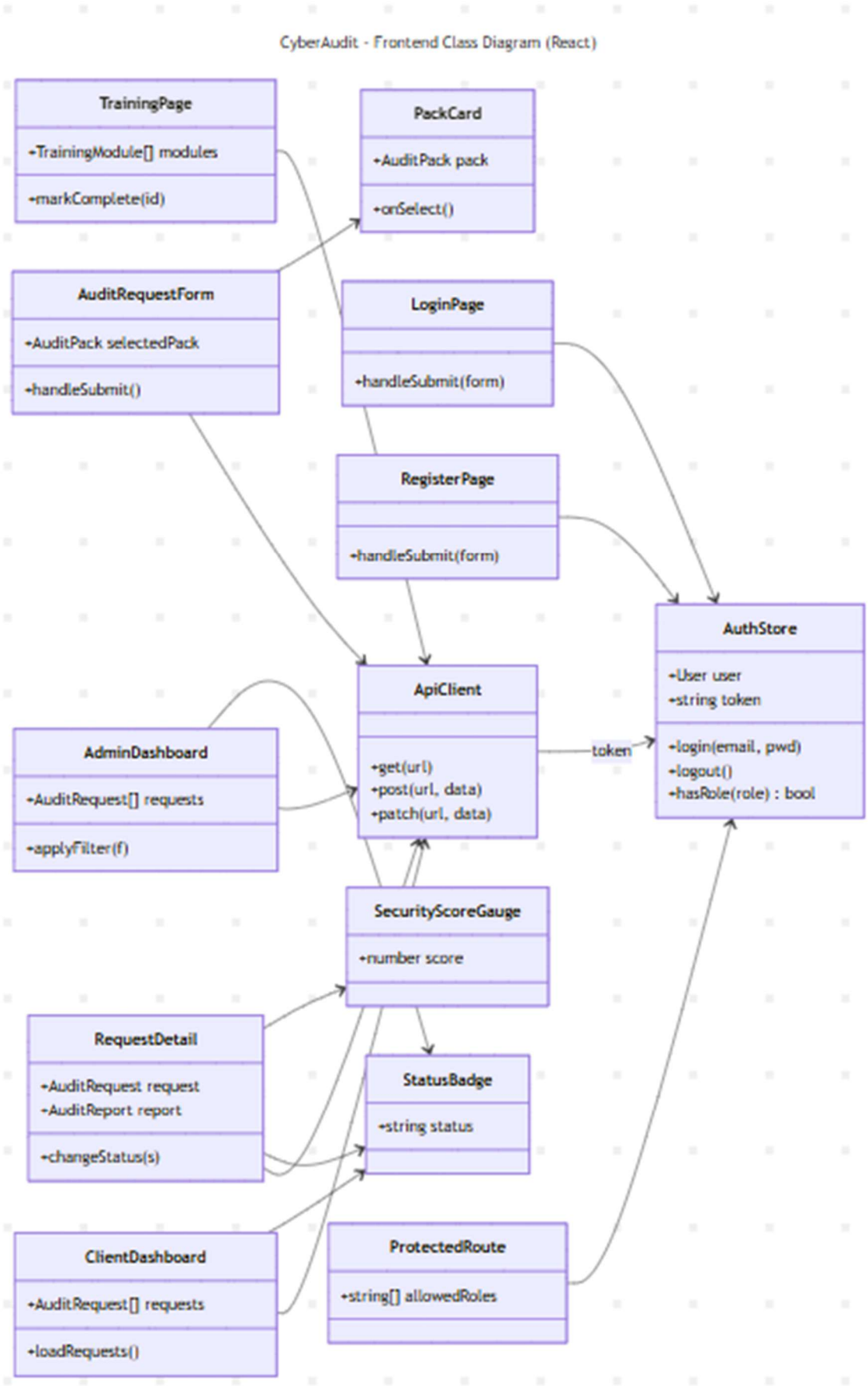
Reusable UI components

Navbar, PackCard, StatusBadge (grey/blue/green/purple), **SecurityScoreGauge** (circular gauge, red < 40, orange 40-70, green > 70), **RequestTable, Pagination, ToastContainer.**

Main Interaction Flow: Request Submission

1. The user logs in via LoginPage → AuthStore stores the JWT.
2. ProtectedRoute authorizes access to the ClientDashboard.
3. Clicking "New Request" → AuditRequestForm loads the packs.
4. Selecting a PackCard → AuditService.create() calls the Django API.
5. The backend creates the AuditRequest and triggers the Celery email task.
6. The client is redirected to RequestDetail (StatusBadge "pending").
7. A notification is sent for each status change on the admin side.

CyberAudit – Frontend Class Diagram (React) Explained



This diagram shows the React frontend architecture for a cybersecurity audit application called **CyberAudit**. It illustrates the main UI components (classes) and how they depend on shared services. Here's a walkthrough.

Central services
Two classes sit at the core of the application and are referenced by almost everything else:
ApiClient is the HTTP layer. It exposes get(url), post(url, data), and patch(url, data) methods that the rest of the app uses to talk to the backend. It reads the auth token from AuthStore (shown by the labeled "token" arrow) so requests are authenticated automatically.

AuthStore holds session state: the current User, the JWT-style token, and methods login(email, pwd), logout(), and hasRole(role): bool. Components consume it to know who's logged in and what they're allowed to do.
Authentication and routing

LoginPage and **RegisterPage** each expose a handleSubmit(form) method. They both depend on ApiClient (to send credentials to the backend) and on AuthStore (to persist the user/token after a successful response).
ProtectedRoute is a route guard with an allowedRoles: string[] property. It queries AuthStore.hasRole(...) to decide whether to render the child route or redirect — this is how the app enforces role-based access.

Client-facing screens
ClientDashboard holds AuditRequest[] requests and a loadRequests() method. It calls ApiClient to fetch the logged-in user's audit requests and uses StatusBadge to render their state.

AuditRequestForm lets a client submit a new audit. It carries a selectedPack: AuditPack and a handleSubmit() method and posts the form to the backend through ApiClient. It also renders **PackCard** components each PackCard displays an AuditPack and exposes an onSelect() callback so the user can pick a pack.

RequestDetail is the per-request view. It holds an AuditRequest and an AuditReport, exposes changeStatus(s), and composes two reusable widgets: **SecurityScoreGauge** (which displays a numeric score) and **StatusBadge** (which renders a status string). It talks to the backend via ApiClient.

TrainingPage shows TrainingModule[] with a markComplete(id) method the training/awareness section of the app, also backed by ApiClient.

Admin side
AdminDashboard mirrors the client dashboard but for administrators. It holds AuditRequest[] requests and an applyFilter(f) method, pulls data through ApiClient, and reuses StatusBadge for display.

How the pieces fit together
The architecture follows a clean layered pattern. Pages and dashboards (top layer) compose small reusable presentational components like PackCard, StatusBadge, and SecurityScoreGauge. They all go through a single ApiClient for backend communication, and a single AuthStore for identity and authorization. ProtectedRoute ties authorization into routing so unauthorized users never reach the protected pages in the first place.
In short: **AuthStore** = who you are, **ApiClient** = how you talk to the server, **ProtectedRoute** = what you're allowed to see, and everything else is feature-specific UI built on top of those three foundations.

Component tree

```
src/
├── components/
│   ├── auth/          Login Form,  RegisterForm, ProtectedRoute
│   ├── dashboard/     ClientDashboard, AdminDashboard, AuditRequestCard, StatusBadge
│   ├── audit/         AuditRequestForm, PackSelector, AuditDetailView
│   ├── training/      ModuleList, ModuleViewer
│   └── shared/        Header, Sidebar, Button, ErrorBoundary HomePage, LoginPage,
├── pages/             RegisterPage, DashboardPage,
│                      NewAuditPage, AuditDetailPage, TrainingPage useAuth,
├── hooks/             useAudits, useApi
├── services/          api.js (axios + JWT), auth/audits/reports services sanitize.js
├── utils/             (DOMPurify), validators.js
└── App.jsx            global routing
```

Conventions : Functional components + hooks (no classes), global state via Context API, sanitize() on all inputs

4.4 Generation of the PDF vulnerability report

The PDF report is the SME client's main deliverable: it consolidates the vulnerability analysis, the security score, and the recommended action plan. Its generation is entirely asynchronous to avoid blocking the admin interface.

Stack technique

- **WeasyPrint:** Converts an HTML+CSS template into a print-quality PDF. Reuses the web's style guide.
- **Celery**: Runs the generate_pdf_task in the background.
- **Redis**: Celery queue (parallel scaling).
- **Stockage**: /media/reports/ in MVP, S3 in V2; access via signed URL (15 min).

Structure of the generated PDF

Section	Contents
Page 1: Cover	CyberAudit logo, client name, case reference (CYB-2026-0042), date, confidentiality
Page 2: Executive Summary	Overall score A to F (scale 0-100) + summary in 3 simplified lines
Page 3: Audited Scope	List of assets analyzed (mail server, Wi-Fi router, workstations, etc.) + EBIOS RM methodology
Pages 4-N: Vulnerabilities	Table findings: severity (Critical/High/Medium/Low), active, description, recommendation
Final page : 30-day action plan	Weekly steps (S1, S2, S3, S4) with concrete actions and priorities
Appendix	Glossary, references (ISO 27005, ANSSI), contact CyberAudit

Full Flow : From Request to Download

1. The admin clicks **Generate PDF report** in AdminRequestDetail.
2. The front calls POST /api/requests/{id}/generate-report with findings.
3. The PdfReportService (Django) creates the AuditReport (summary, score, findings JSON).
4. It dispatches generate_pdf_task.delay(report_id) and returns 202 Accepted.
5. The Celery worker renders report.html with the data; WeasyPrint produces the PDF.
6. File saved to /media/reports/CYB-2026-0042.pdf, pdf_path updated.
7. A report_ready Notification is created, email sent via send_email_task.
8. Client-side, ClientDashboard shows **Download report** → GET /api/reports/{id}/download streaming.

PDF Access Security

- **JWT required** on /api/reports/{id}/download. No anonymous access.
- **IsAdminOrOwner permission** : 403 Forbidden otherwise.
- **Signed URL** with timestamp, expires after 15 minutes.
- **No browser cache** : Cache-Control, no-store, private.
- **Audit log** (V2) : who, when, from which IP, GDPR-compliant traceability.

Generation Failure Handling

- **Auto-retry** : autoretry_for=(Exception,), max_retries=3, exponential backoff (10s, 30s, 90s).
- **Intermediate status** : pdf_path empty until generated → front shows "Generating…".
- **Failure notification** : after 3 failures, generation_failed notification sent to admin.
- **Manual regeneration** via the **Generate PDF report** button.

4.5 Monitoring & Observability

The application must remain traceable in production: we must know *who did what, when, and why it crashed*. The strategy rests on 3 pillars: structured logs, error monitoring, alerting.

4.5.1 Structured Logs (JSON)

Django configures structlog to produce JSON logs consumable by Railway and a future ELK/Loki stack. Each log carries at minimum: timestamp, level, request_id (correlation), user_id, endpoint, latency_ms.


```
{
  "timestamp": "2026-05-18T09:24:11Z",
  "level": "INFO",
  "event": "audit_request.created",
  "request_id": "req-7f2a...",
  "user_id": "8f2a...c9",
  "audit_id": "CYB-2026-0042",
  "pack": "security",
  "latency_ms": 142
}
```

4.5.2 Error Monitoring : Sentry

- **Back-end**: sentry-sdk[30jango,celery] captures all Django and Celery exceptions with stack trace, user context, request payload (PII anonymized).
- **Front-end**: @sentry/react captures JS errors and unhandled promise rejections, plus session replay on error.
- **Release tracking**: every git tag pushes a Sentry release → per-version regression tracking.
- **Sample rate**: 100% of errors on MVP; 10% of performance transactions to stay within the free quota.

4.5.3 Business & Technical Metrics

Metric	Source	Alert Threshold
API response time (p95)	Sentry Performance / Railway metrics	> 800 ms for 5 min
5xx error rate	Sentry	> 1% over 10 min
Consecutive Celery failures	Structured logs	≥ 3 on the same task
Active DB connections	Railway Postgres metrics	> 80% of pool
/media disk space	Railway	> 85%
Audit requests / day	Django + admin dashboard	Growth metric, no alert

4.5.4 Alerting

- **Sentry** emails the tech lead on every new unseen error, and a Slack DM (V2) if error rate > threshold.
- **Healthcheck**: GET /api/health/ endpoint checks DB, Redis, SMTP. Monitored by UptimeRobot (free) every 5 min.
- **Celery failure notification**: after 3 retries, an admin email is sent (already documented in §4.4).

4.5.5 Application Audit Log (V2)

An AUDIT_LOG table tracks sensitive actions: logins, report downloads, status changes, account deletions. Fields: id, user_id, action, resource, ip, user_agent, created_at. Retention 1 year, GDPR-compliant (see §7.5).

4.6 Database Migrations & Seeding

Schema version transitions and pre-population of reference data (4 audit packs, initial training modules, admin account) are fully automated. No manual intervention in production.

4.6.1 Django Migrations Strategy

- **Convention**: 1 migration = 1 atomic change. Explicit name: 0007_add_security_score_to_auditreport.py.
- **Schema migrations**: generated via python manage.py makemigrations, never hand-edited.

- **Data migrations**: use migrations.RunPython(forward, reverse) with a systematic reverse function to allow rollback.
- **CI enforcement**: python manage.py migrate --check in the pipeline to detect non-generated migrations.
- **Production**: Railway runs migrate automatically on deploy via the Procfile release command.

4.6.2 Example Data Migration

```
# audits/migrations/0002_seed_packs.py
from django.db import migrations

def seed_packs(apps, schema_editor):
    AuditPack = apps.get_model('audits', 'AuditPack')
    AuditPack.objects.bulk_create([
        AuditPack(code='audit',      name='Audit',      duration_days=5,  price=490),
        AuditPack(code='security',   name='Security',   duration_days=10, price=990),
        AuditPack(code='protection', name='Protection', duration_days=15, price=1490),
        AuditPack(code='premium',    name='Premium',    duration_days=20, price=2490),
    ])

def remove_packs(apps, schema_editor):
    apps.get_model('audits', 'AuditPack').objects.all().delete()

class Migration(migrations.Migration):
    dependencies = [('audits', '0001_initial')]
    operations = [migrations.RunPython(seed_packs, remove_packs)]
```

4.6.3 Fixtures (Demo Data)

JSON fixtures in backend/fixtures/ load demo data (5 fake clients, 10 audit requests, 3 reports) for jury demos and E2E tests:

```
python manage.py loaddata fixtures/demo_clients.json
python manage.py loaddata fixtures/demo_audits.json
python manage.py loaddata fixtures/demo_reports.json
```

A wrapper script scripts/seed_demo.sh chains migrate + loaddata for a 1-command demo environment.

4.6.4 Initial Admin Creation

No plaintext passwords in code. A custom Django command create_initial_admin reads ADMIN_EMAIL and ADMIN_PASSWORD from environment variables and creates the user if the table is empty:

```
python manage.py create_initial_admin
# Idempotent: does nothing if an admin already exists
```

4.6.5 Rollback Strategy

Scenario	Procedure
Broken migration just after deploying	python manage.py migrate audits 0006 reverts to the previous migration. The RunPython reverse function executes.
Corrupted data detected	Restore the latest Railway backup (1-click), then investigate off-prod on a snapshot.
Migration requiring downtime	Planned maintenance window, Vercel maintenance page enabled, migration executed, verification, reopening.
Non-backward-compatible schema change	"Expand & contract" strategy: 1) add new column, 2) dual-write in app, 3) backfill, 4) switch reads, 5) drop old column

4.6.6 Snapshot Before Each Release

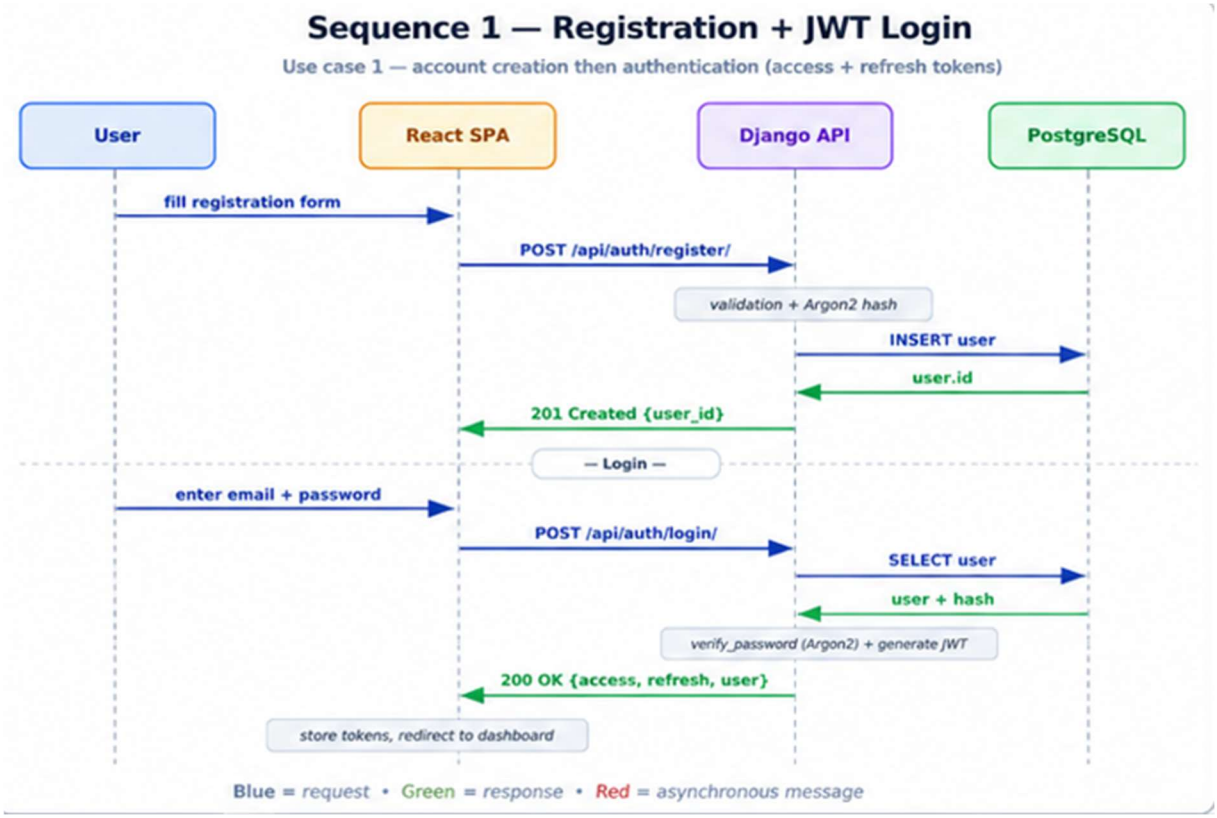
Before each merge to main (prod release), a manual Railway DB snapshot is triggered via railway db snapshot create. 14-day retention. Allows rollback in < 5 min in case of major issue.

Sequence Diagrams

5.1 Sign-up + JWT Login

The user enters email and password. The front calls POST /api/auth/register/, then POST /api/auth/login/. Django validates credentials, issues access + refresh tokens and returns them. The front stores them in AuthStore and redirects to the role-appropriate dashboard.

Figure 5: Sign-up + JWT Login Sequence



This diagram illustrates **Use Case 1: Registration then Login** for a web application, using JWT (JSON Web Tokens) for authentication. It shows how four actors interact over time: the **User**, the **React SPA** (Single Page Application, the frontend), the **Django API** (the backend), and the **PostgreSQL** database.

Messages Exchanged (Detailed)

Marie → **RegisterPage**: fills out the form (email, password, company_name).

RegisterPage → **ApiClient**: POST /api/auth/register/ with JSON body {email, password, company_name, company_size, sector}.

ApiClient → **Django (RegisterView)**: DRF serializer validates fields (unique email, password strength).

Django → **PostgreSQL**: INSERT INTO accounts_user with bcrypt-hashed password.

Django → **Celery**: send_welcome_email.delay(user_id).

Django → **RegisterPage**: 201 Created with {user, access, refresh}.

RegisterPage → **AuthStore**: stores accessToken and refreshToken.

RegisterPage → **React Router**: redirect to /dashboard based on user.role.

Celery worker → **SMTP Gmail**: sends welcome email (asynchronous, off the HTTP thread).

5.2 Audit Request Submission (async)

The client submits the audit form. The front calls POST /api/audits/ with the JWT in headers. Django validates via the DRF serializer, persists to the database, enqueues a Celery acknowledgment-email task, and responds 201 Created. The Celery worker pops the task from Redis and sends the email via SMTP (EmailJS).

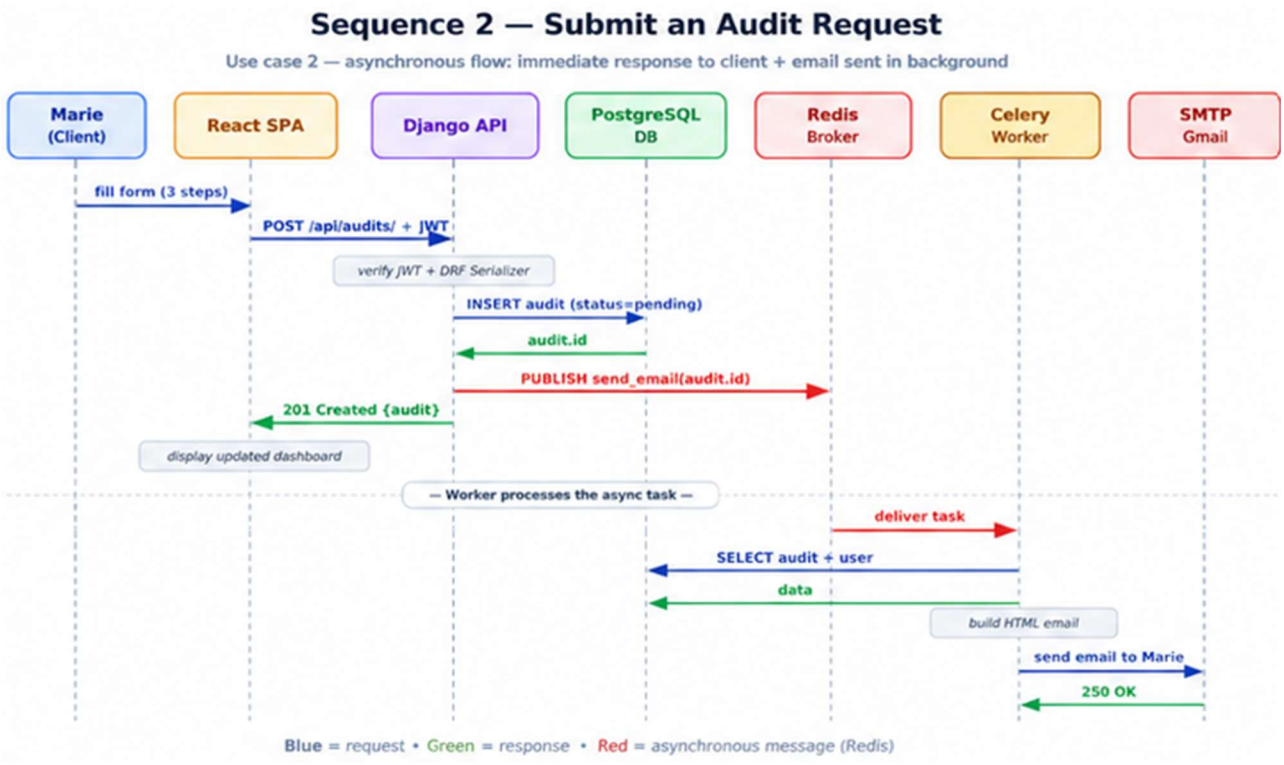


Figure 6: Async Audit Submission Sequence

Messages Exchanged (Detailed)

Marie → AuditRequestForm: selects a pack and enters `scope_notes`.

AuditRequestForm → ApiClient: `POST /api/audits/`, header `Authorization: Bearer <JWT>`, body `{pack_id, scope_notes}`.

ApiClient → Django (AuditCreateView): middleware checks JWT, DRF validates fields.

Django → PostgreSQL: `INSERT INTO audits_auditrequest` with `status='pending'`, `reference='CYB-2026-XXXX'`.

Django → Redis (Celery broker): enqueues `send_acknowledgment_email.delay(audit_id)`.

Django → AuditRequestForm: `201 Created` with request details.

AuditRequestForm → ClientDashboard: redirect with "Request sent" toast.

Celery worker ← Redis: dequeues the task.

Celery worker → SMTP Gmail: sends acknowledgment to Marie + notification to Karim.

Celery worker → PostgreSQL: `UPDATE notifications SET status='sent'`.

5.3 PDF Report Generation (admin)

The admin clicks **Generate PDF report**. The front calls POST /api/audits/{id}/generate-report/. Django creates the AuditReport and enqueues the Celery task, returning 202 Accepted. The Celery worker renders the HTML template, WeasyPrint produces the PDF, the file is stored, and an email notification is sent to the client.

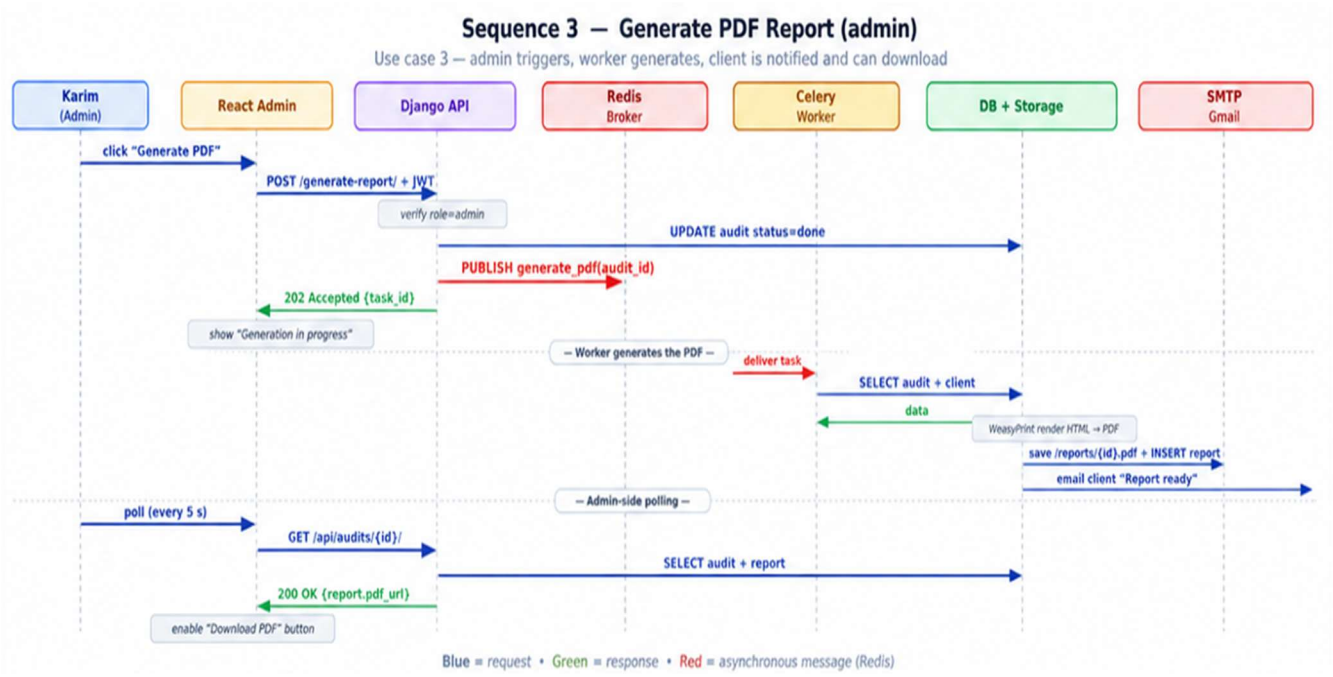


Figure 7: PDF Report Generation Sequence

Messages Exchanged (Detailed)

Karim → AdminRequestDetail: clicks "Generate PDF report" with entered findings.

AdminRequestDetail → ApiClient: `POST /api/audits/{id}/generate-report/`, body `{findings, summary, security_score}`.

ApiClient → Django (PdfReportService): checks `IsAdmin` permission, verifies `status='done'`.

Django → PostgreSQL: `INSERT INTO reports_auditreport` with `pdf_path=NULL` (in progress).

Django → Redis: enqueues `generate_pdf_task.delay(report_id)`.

Django → AdminRequestDetail: `202 Accepted` with `{report_id, status: "generating"}`.

Celery worker ← Redis: dequeues the task.

Celery worker → Django templates: renders `report.html` with context (findings, score, recommendations).

Celery worker → WeasyPrint: converts HTML+CSS to binary PDF.

Celery worker → Storage: saves to `/media/reports/CYB-2026-0042.pdf`.

Celery worker → PostgreSQL: `UPDATE auditreport SET pdf_path=...`.

Celery worker → SMTP Gmail: sends "Report available" email to Marie.

Marie → ClientDashboard: sees "Download report" link appear (30 s polling).

Marie → Django: `GET /api/audits/{id}/report/` with JWT.

Django → Marie: 15-min signed URL, PDF streaming.

API Documentation

6.1 External APIs Consumed

External API	Usage	Reason
SMTP Gmail	Transactional emails (sign-up confirmation, status change, report delivery)	Free up to 500 emails/day, simple, reliable. Migration to SendGrid/Mailgun possible in V2.
Let's Encrypt	Automatic HTTPS certificates	Free, natively integrated in Vercel and Railway.

6.2 Internal API : REST Endpoints

Convention: prefix /api/, JSON for requests and responses, JWT in Authorization: Bearer <token>, standard HTTP codes.

6.2.1 Authentication : /api/auth/

Method	URL	Auth	Description	Body	2xx Response	Errors
POST	/api/auth/register/	—	Create client account	{email, password, company_name, company_size, sector}	201 {user, access, refresh}	400, 409 email exists
POST	/api/auth/login/	—	Login + JWT tokens	{email, password}	200 {access, refresh, user}	401, 429 rate limit
POST	/api/auth/refresh/	—	Refresh access token	{refresh}	200 {access}	401 refresh expired
POST	/api/auth/logout/	✓	Invalidate refresh	{refresh}	204	401
POST	/api/auth/password-reset/	—	Send reset email	{email}	200 (always, anti-leak)	—

6.2.2 Audit Requests : /api/audits/

Method	URL	Auth	Role	Describe
GET	/api/audits/	✓	client / admin	Paginated list (client → own / admin → all)
POST	/api/audits/	✓	client	Create a new request
GET	/api/audits/{id}/	✓	owner / admin	Request detail
PATCH	/api/audits/{id}/	✓	admin	Edit status or notes
DELETE	/api/audits/{id}/	✓	admin	Archive (soft delete)
POST	/api/audits/{id}/generate-report/	✓	admin	Trigger PDF generation (Celery)
GET	/api/audits/{id}/report/	✓	owner / admin	Download the PDF (binary)

Example ; Creating a Request

```
POST /api/audits/
Authorization: Bearer eyJhbGc...
Content-Type: application/json

{ "pack_id": 2, "scope_notes": "Audit of file server + 12 Windows workstations." }

HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "8f2a...c9",
  "client": "user-uuid",
  "pack": {"id": 2, "name": "standard"},
  "status": "pending",
  "scope_notes": "Audit of file server ...",
  "created_at": "2026-05-18T09:24:11Z"
}
```

6.2.3 Audit Packs : /api/packs/

Method	URL	Auth	Description
GET	/api/packs/	—	Public list of the 4 packs
GET	/api/packs/{id}/	—	Pack detail

6.2.4 Training : /api/training/

Method	URL	Auth	Description
GET	/api/training/modules/	✓	List available modules
GET	/api/training/modules/{id}/	✓	Module content
POST	/api/training/modules/{id}/start/	✓	Mark as started

POST	/api/training/modules/{id}/complete/	✓	Mark as completed

6.2.5 User Profile : /api/users/

Method	URL	Auth	Description
GET	/api/users/me/	✓	Current user's profile
PATCH	/api/users/me/	✓	Update own profile
POST	/api/users/me/change-password/	✓	Change own password
DELETE	/api/users/me/	✓	Delete own account (GDPR)

6.3 Cross-Cutting Conventions

- **Pagination** : `?page=1&page_size=20`, response `{count, next, previous, results}`
- **Filtering** : `?status=pending&ordering=-created_at`
- **Errors** : unified format `{error: "code", message: "Message", details: {...}}`
- **Rate limiting** : 5 req/min on `/auth/login/` and `/auth/password-reset/` (IP-based)
- **Versioning** : no v1 in URL on the MVP; in V2 if breaking changes → `/api/v2/`

SCM & QA Strategy

7.1 Source Code Management (Git)

Host: GitHub repo “Tommy-JOUHANS/portfolio” (private until defense, then public).

Branching Strategy (lightweight Git Flow)

main ← production only (protected, Vercel/Railway continuous deploy)
develop ← integration branch (CI required)
feature/<name> ← new features (1 PR = 1 feature)
fix/<name> ← bug fixes
hotfix/<name> ← urgent fixes directly on main
release/<v> ← stabilization before release (from W11)

Commit Convention : Conventional Commits

feat(audits): add request form
fix(auth): fix expired JWT refresh token
docs(readme): update technical stack
test(audits): add CRUD unit tests
chore(deps): bump Django 5.0.2 → 5.0.3
refactor(reports): extract PDFGenerator service

Pull Request Rules

Rule	Detail
Required review	Each PR validated by the other member before merge
Green CI	Tests + lint OK before merge
No direct push	main and develop protected by GitHub Branch Protection
Structured description	"What changes / Why / How to test"
Squash merge	1 PR = 1 commit in develop (clean history)

7.2 QA Strategy

Test Pyramid

```
          /\
         /E2E\         5 %  – Cypress (critical journeys)
        /------\
       / Integ. \ 20 %  – API + DB integration (Django + DRF)
      /----------\
     /  Unit tests \ 75 % – Jest (front) + pytest (back)
    /----------------\
```

Chosen Tools

Layer	Tool	Target
Back-end unit	pytest + pytest-django	Models, services, serializers
Back-end API	pytest + APIClient (DRF)	auth, audits, reports endpoints
Front-end unit	Jest + React Testing Library	Components, hooks, utils
E2E	Cypress	Sign-up → request → email (mock)
Security	Postman (collection) + bandit	XSS, SQLi, brute-force, JWT tampering
UX	3 user sessions (W7-W10)	Marie persona, < 3-min flow
Coverage	coverage.py + jest --coverage	Target: 80% min on critical apps
Lint	ruff + black ; eslint + prettier	Pre-commit hook
Type checking	mypy ; propTypes or TS (V2)	From W4

7.3 Manual Security Tests (W7-W10)

Test	Method	Expected Result
Persistent XSS	<script>alert(1)</script> in scope_notes	Escaped / sanitized
SQL injection	' OR 1=1 -- in login email	No leak, ORM bind
Brute-force	100 login attempts	Blocked from 6th (rate-limit)
JWT tampering	Modify role in payload	Invalid signature → 401
CSRF	POST without Authorization header	401
HSTS / CSP	HTTP headers in prod	Present and strict

7.4 CI/CD Pipeline

GitHub Actions

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-python 3.11
      - install: pip install -r backend/requirements-dev.txt
      - lint:    ruff check . && black --check .
      - test:    pytest --cov=apps --cov-fail-under=80

  frontend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-node 18
      - install: npm ci
      - lint:    npm run lint
      - test:    npm test -- --coverage
      - build:   npm run build
```

Continuous deployment:

Environment	Trigger	Target	URL
Preview	All PR	Vercel preview + Railway branch deployment	pr-XX.preview.cyberaudit.app
Staging	Merge on develop	Vercel staging + Railway staging	staging.cyberaudit.app
Production	Merge on main (release tag)	Vercel + Railway prod	cyberaudit.app

Release strategy:

• SemVer versioning: 0.1.0 (W4 partial MVP) → 0.5.0 (W6 async) → 1.0.0 (W12 presentation)
• Annotated git tag for each release: git tag -a v1.0.0 -m "MVP presentation Holberton"
• Changelog kept up to date in CHANGELOG.md

Appendices

8.1 Glossaire

Term	Definition
MVP	Minimum Viable Product : minimal deliverable version of the product
JWT	Web Token : stateless authentication standard
MoSCoW	Prioritization method: Must / Should / Could / Won't
CRUD	CRUD Create / Read / Update / Delete
ORM	ORM Object Relational Mapping : database abstraction
CI/CD	CI/CD Continuous Integration / Continuous Deployment
GDPR	GDPR General Data Protection Regulation
HSTS HTTP	Strict Transport Security : anti-downgrade header
CSP	Content Security Policy : anti-XSS header
SemVer	Semantic Versioning : MAJOR.MINOR.PATCH
SPA	Single Page Application

8.2 Mapping Stage 3 ↔ RNCP 5 skills

RNCP Web and Mobile Web Developer (TP issued by the Ministry of Labor). This document covers the following blocks:

RNCP Block	Skill	Documentation section
BC01 : Develop the front-end part	Mock up an application	§2.3 (mockups)
BC01	Create static and dynamic interfaces	§4.3 (Component Responses)
BC01	Develop the front-end part	§3 (Architecture), §6 (API)
BC02 : Develop the back-end part	Create a database	§4.1 (Entity Relationship diagram)
BC02	Develop data access components	§4.2 (Django Template)
BC02	Develop the back-end part	§6 (REST API)
BC02	Design and implement components in a content management or e-commerce application	§4.2 (Services), §5 (Sequences)
Transversal	Work in an agile team / Versioning	§7 (SCM/QA)

8.3 Technical sources & references

Reference	Links
Django REST Framework : Documentation	https://www.django-rest-framework.org
SimpleJWT : JWT for Django	https://django-rest-framework-simplejwt.readthedocs.io
Celery : Distributed Task Queue	https://docs.celeryq.dev
WeasyPrint : HTML to PDF	https://weasyprint.org
OWASP Top 10 :security repository	https://owasp.org/www-project-top-ten
Conventional Commits	https://www.conventionalcommits.org
RNCP 38038 : Web and Mobile Web Developer	https://www.francecompetences.fr/recherche/rncp/38038
ANSSI : Overview of the cyber threat 2023	https://www.ssi.gouv.fr

8.4 Internal links within the project

- Stage 1: Project Charter (FR)
- Stage 2: Planning & Gantt (FR)
- Figma Wireframes
- MVP Code Repo

8.5 Internal project links

- Stage 1 : Project Charter (FR)
- Stage 2 : Planning & Gantt (FR)
- Figma Wireframes to be published in W1 by Tommy

Document prepared as part of Stage 3: Technical Documentation, Holberton School Dijon, class of 2026. Designed to be reused as-is in the RNCP 5 certification folder “Web and Mobile Web Developer”.